

LOCAL TRAJECTORY OPTIMIZATION

FOR LUNAR ROVER ON SOUTH POLE

A Terrain-Based Navigation Simulation Using DEM Data and Real-Time 3D Visualization

Research Report | Academic Year 2025–2026

ABSTRACT

This report presents the design and implementation of a local path optimization system for a simulated lunar rover operating on the Moon's South Pole terrain. Real elevation data was obtained from NASA's Lunar Reconnaissance Orbiter Camera (LROC) Digital Elevation Model (DEM) repository. The DEM data was processed through QGIS for raster layer extraction and terrain preprocessing. The resulting heightmap was subsequently imported into Unity 3D to construct a high-fidelity terrain environment, upon which an A pathfinding algorithm was deployed to enable autonomous and slope-aware rover navigation. The simulation incorporates real-time telemetry display, slope hazard detection, and interactive path visualization, providing a robust platform for evaluating rover traversal strategies in lunar polar environments.*

Table of Contents

1. Project Introduction & Objectives	3
2. System Architecture Overview	4
3. Unity Environment Setup	5
4. NASA Dataset — Detailed Description	7
4.1 LOLA DEM — Elevation & Slope Data	7
4.2 LROC NAC DTM — Terrain Camera Data	10
4.3 Zenodo Crater Catalog	12
4.4 Bullialdus M3 — Mineral Spectral Data	14
5. Terrain Surface — WAC Mosaic Texture	16
6. Rover Model & Controller	18
7. Sensor System (LiDAR, Radar, Mineral)	20
8. A* Pathfinding Algorithm	22
9. Machine Learning Pipeline	24
9.1 Data Generation (Synthetic Dataset)	24
9.2 Feature Engineering	26
9.3 Model Architecture (DNN + ResBlocks)	27
9.4 Training Process	29
9.5 ONNX Export & Unity Sentis Inference	31
10. Obstacle Avoidance System	33
11. Camera System	35
12. Mistakes Made & How They Were Solved	36
13. Results & Analysis	40
14. Future Work & Improvements	42
15. Conclusion	44

1. Project Introduction & Objectives

This project is a comprehensive simulation of a Lunar South Pole Rover built inside Unity 6 using Universal Render Pipeline (URP). The goal was to create a realistic, AI-powered rover capable of autonomously navigating the treacherous terrain of the lunar south pole — a region of immense scientific interest due to the presence of permanently shadowed craters believed to contain water ice, helium-3 deposits, and various mineral resources.

The project integrates real NASA data — including LOLA Digital Elevation Models (DEM), LROC NAC camera terrain maps, and Chandrayaan-1 mineral spectral data — into a functioning Unity simulation. An A* pathfinding algorithm plans routes across the terrain, while a custom Multi-Task Deep Neural Network trained on 1,00,000 data points provides real-time terrain classification and slope prediction using simulated LiDAR, radar, and mineral sensor data.

Project Objectives

- Create a high-fidelity lunar terrain using real NASA LOLA DEM heightmap data
- Apply real WAC mosaic surface texture to match actual lunar appearance
- Implement a functional rover with WASD manual control and autonomous A* navigation
- Build a sensor system simulating LiDAR (72 rays), radar, and mineral detection
- Train a custom DNN using PyTorch on 1,00,000 synthetic data points derived from real NASA data
- Deploy the trained ONNX model in Unity using Sentis for real-time AI inference
- Develop an intelligent obstacle avoidance system combining LiDAR and ML predictions
- Implement a dual-camera system for both overview and rover-perspective views

Technologies Used

Technology	Version	Purpose
Unity	6 (URP)	3D simulation environment
C#	Latest	Unity scripting
Python	3.10	ML training pipeline
PyTorch	2.x	Neural network training
Unity Sentis	2.x	ONNX model inference in Unity
ONNX	opset 12	Universal model export format
NASA LOLA DEM	87S 5m/pixel	Lunar elevation data
LROC NAC	Derived DTM	High-res terrain camera data

Technology	Version	Purpose
Zenodo LU5M	5M craters	Lunar crater catalog
Chandrayaan-1 M3	Bullialdus region	Mineral spectral data
WAC Mosaic	Full south pole	Surface texture
A*	Custom C#	Path planning algorithm

2. System Architecture Overview

The system follows a layered architecture where each component feeds into the next. Real NASA data forms the foundation, Unity provides the simulation environment, and the ML model provides intelligent decision-making on top.

Architecture Layers

Layer	Components	Description
Data Layer	LOLA DEM, NAC DTM, Crater CSV, M3 Mineral	Real NASA data providing ground truth for terrain, craters, and minerals
Environment Layer	Unity Terrain, WAC Texture, Lighting	Visual representation of the lunar surface
Sensor Layer	LiDAR (72 rays), Radar, Mineral Sensor	Simulated rover sensors collecting environmental data
ML Layer	PyTorch DNN, ONNX Export, Unity Sentis	Trained neural network predicting terrain class and slope
Navigation Layer	A* Pathfinder, PathVisualizer	Route planning from start to destination
Control Layer	RoverController, Obstacle Avoidance	Rover movement, terrain alignment, danger response
Display Layer	HUD, Camera System, Visualization	Mission control display and camera views

Data Flow

```
NASA Data -> Python Processing -> Unity Terrain
Unity Terrain -> Sensor Scripts -> 78 Feature Vector
78 Features -> ONNX Model (Sentis) -> Slope + Terrain Class
Terrain Class + LiDAR -> Obstacle Decision -> Rover Action
A* Algorithm -> Safe Path -> Rover Navigation
```

3. Unity Environment Setup

The Unity environment was set up using Unity 6 with Universal Render Pipeline (URP). URP was chosen for its better performance characteristics and modern rendering features suitable for a space simulation. The setup involved configuring terrain, lighting, camera systems, and all the necessary scripts.

3.1 Terrain Setup

The terrain was created using Unity's built-in Terrain system. A LOLA DEM heightmap (1024x1024 pixels cropped from the full 87S dataset) was imported and applied to the terrain. The terrain size was set to match the scale of the real lunar south pole area being simulated.

Key Terrain Settings

Parameter	Value	Reason
Terrain Size X	1000m	Covers sufficient area for rover navigation
Terrain Size Z	1000m	Square terrain for uniform coverage
Terrain Height	600m	Accommodates lunar elevation variation
Heightmap Resolution	1025x1025	Unity standard for heightmaps
Base Map Distance	1000	Full quality texture rendering distance
Pixel Error	5	Balance between quality and performance

3.2 Lighting Setup

Lunar lighting is unique — there is no atmospheric scattering, resulting in harsh shadows and high contrast. The Directional Light was configured to simulate the sun at a low angle typical of the south pole, where the sun never rises high above the horizon.

- Directional Light Intensity: 1.5-2.5 (adjusted based on scene needs)
- Light Rotation: X=45, Y=-30 (low sun angle for south pole)
- Shadow Type: Soft shadows for realism
- Ambient Mode: Color set to dark grey (no sky in space)
- Global Volume: Post-processing for space-like appearance

3.3 Problems During Unity Setup

Problem: Initial terrain appeared completely black — lighting was too dim. Solution: Increased Directional Light intensity to 2.0 and changed Ambient Color from black to grey (R=80, G=80, B=80).
--

Problem: Terrain texture showed repeated tiles creating ugly grid pattern. Solution: Changed Tiling Size from 100,100 to 1,1 so the WAC texture covers the entire terrain exactly once without repeating.

Problem: Blue sky appeared in rover camera view instead of black space. Solution: Changed camera Background Type from Skybox to Solid Color and set to dark grey matching the game view background.

4. NASA Dataset — Detailed Description

Four distinct NASA and scientific datasets were used in this project. Each dataset serves a specific purpose in the simulation and ML training pipeline. Understanding these datasets is crucial to understanding how the AI makes decisions.

4.1 LOLA DEM — Lunar Orbiter Laser Altimeter

The Lunar Orbiter Laser Altimeter (LOLA) is an instrument aboard NASA's Lunar Reconnaissance Orbiter (LRO), launched in June 2009. LOLA uses a laser that fires 28 times per second, splitting into 5 beams to measure the precise distance to the lunar surface. By accumulating billions of these measurements over years of operation, LOLA has produced the most detailed topographic map of any planetary body in our solar system.

What is LOLA DEM?

A Digital Elevation Model (DEM) is a 3D representation of terrain elevation. The LOLA DEM used in this project (ldem_87s_5mpp) covers the lunar south pole region centered at 87 degrees South latitude. The '5mpp' designation means 5 meters per pixel — each pixel in the image represents a 5x5 meter square on the lunar surface.

File Details

Property	Value
Filename	ldem_87s_5mpp.tif
Format	GeoTIFF (.tif)
Coverage	Lunar South Pole (87 degrees South)
Resolution	5 meters per pixel
Original Size	40,000 x 39,998 pixels
File Size	~6 GB (raw), ~4 MB after cropping
Elevation Range	-8,200m to -1,200m (below datum)
Data Type	16-bit signed integer (elevation in meters)
Source	NASA PDS Geosciences Node
Instrument	LRO/LOLA (Lunar Orbiter Laser Altimeter)
Mission	Lunar Reconnaissance Orbiter (LRO)

Scientific Significance

The lunar south pole is one of the most scientifically important regions on the Moon. LOLA data revealed that this area contains permanently shadowed regions (PSRs) — craters so

deep that sunlight never reaches their floors. These PSRs can reach temperatures as low as -173 degrees Celsius, cold enough to trap volatile compounds including water ice over billions of years. NASA's Artemis program targets this region for human return to the Moon.

How LOLA Data Was Used in This Project

- Heightmap: The DEM was converted to a grayscale PNG and applied to the Unity terrain as a heightmap, creating realistic crater-filled lunar topography
- Slope Calculation: Python numpy was used to calculate the gradient (slope angle) at every pixel using the formula: $\text{slope} = \arctan(\sqrt{dx^2 + dy^2})$
- Training Labels: Slope values served as the primary regression label for the ML model — the model learns to predict slope from sensor data
- Obstacle Zones: Pixels with slope > 20 degrees were marked as non-traversable obstacles in the A* pathfinding grid
- Water Ice Identification: Pixels with very low elevation (< -8500m) in the south pole region were labeled as potential Water Ice zones in training data
- Radar Simulation: The height values were used to simulate the rover's downward-facing radar sensor

Processing Steps

1. Downloaded Idem_87s_5mpp.tif from NASA PDS Geosciences Node
2. Attempted to load full file in Python — failed due to 6GB size causing memory error
3. Used rasterio with Window cropping to load only the center 2000x2000 pixels
4. Applied numpy gradient calculation to derive slope at every point
5. Saved as dem.npy and slope.npy (numpy arrays) for fast access during dataset generation
6. Slope values used directly as training labels for the ML regression task

Key Insight: LOLA reveals that the lunar south pole has average slopes of 4-22 degrees with extreme craters reaching 45+ degrees. The permanently shadowed crater interiors represent the coldest known places in the solar system.

Problems with LOLA Data

Problem: Loading the full 40,000 x 39,998 pixel file caused a memory error: 'Unable to allocate 5.96 GiB for array'. Solution: Used rasterio's Window class to load only the center 2,000 x 2,000 pixels, reducing memory usage from 6GB to 16MB.

Problem: After loading, slope calculation on the full array caused another allocation error. Solution: Cropped the DEM first, then calculated slope on the smaller array. Final dem.npy = 3.9MB.

4.2 LROC NAC DTM — Lunar Reconnaissance Orbiter Camera

The Lunar Reconnaissance Orbiter Camera (LROC) consists of three cameras: two Narrow Angle Cameras (NAC) and one Wide Angle Camera (WAC). The NAC cameras have a resolution of 0.5 meters per pixel — detailed enough to see individual boulders and astronaut footprints from orbit. The Digital Terrain Map (DTM) products are derived from stereo image pairs taken by the NAC cameras.

What is NAC DTM?

NAC DTMs are high-resolution elevation models derived from stereo photogrammetry. By comparing two images of the same area taken from slightly different viewing angles, scientists can calculate precise three-dimensional terrain models. These products reveal surface details far beyond what LOLA laser altimetry can capture alone.

File Details

Property	Value
Dataset Type	PDS4 Special Data Products - NAC Digital Terrain Map (SDNDTM)
Format	GeoTIFF (.tif)
Resolution	0.5 to 2 meters per pixel
Coverage	Selected high-interest areas (Apollo sites, south pole regions)
File Size	Hundreds of MB to several GB per tile
Data Content	Elevation, slope, roughness
Source	NASA PDS / LROC website (lroc.syr.edu)
Instrument	LRO/LROC Narrow Angle Camera
Products Available	660 individual terrain map tiles

How NAC DTM Was Used

- Radar Sensor Data: NAC DTM height values were used to simulate the rover's downward-facing radar sensor readings
- High-Resolution Detail: Provides finer terrain detail than LOLA for areas where the rover is operating
- Secondary Elevation Source: Cross-referenced with LOLA data to improve accuracy
- Saved as nac.npy: 732MB numpy array storing height values for the simulated area

Problems with NAC Data

Problem: The downloaded NAC DTM file had a different name than expected (e.g., NAC_DTM_APOLLO16.tif vs the expected nac_dtm.tif), causing a file-not-found error. Solution:

Used Python's glob module to automatically find any .tif file in the raw data folder that was not the LOLA or Bullialdus file.

Problem: NAC file was 732MB after conversion to numpy, making it impractical to load fully.
Solution: Used rasterio Window cropping to load only the relevant portion matching the LOLA crop area.

4.3 Zenodo Crater Catalog — LU5M812TGT

The Zenodo crater catalog used in this project (LU5M812TGT) is a machine learning-derived catalog of over 5 million lunar craters. Unlike traditional crater catalogs that require laborious manual identification by geologists, this catalog was created using deep learning algorithms trained on LROC imagery to automatically detect and measure craters across the entire lunar surface.

What is the LU5M Crater Catalog?

This catalog represents the most comprehensive mapping of lunar impact craters ever produced. The original Robbins (2018) catalog identified approximately 1.3 million craters manually. The LU5M catalog extends this by using AI to identify smaller craters (as small as 1km in diameter) that were previously unmapped. The catalog contains precise coordinates, diameters, and confidence scores for each detected crater.

File Details

Property	Value
Filename	craters.csv (downloaded from Zenodo record 13990480)
Format	CSV (Comma-Separated Values)
Total Craters	5,000,000+ entries
File Size	422 MB
Key Columns	Latitude, Longitude, Diameter_km, Confidence
Diameter Range	1 km to 2,500 km
Coverage	Full lunar surface (global)
Creation Method	Deep learning on LROC imagery
Source	Zenodo (zenodo.org/records/13990480)

Scientific Significance

Impact craters are the dominant geological feature on the Moon. The size-frequency distribution of craters is used to date planetary surfaces — older surfaces have more craters. The lunar south pole craters are particularly significant because many are permanently shadowed, creating cold traps that may have accumulated water ice delivered by comets and asteroids over billions of years. NASA's LCROSS mission (2009) confirmed water ice in the permanently shadowed Cabeus crater.

How Crater Data Was Used

- Obstacle Identification: Crater locations were used to identify danger zones in the A* pathfinding grid
-

-
- Training Labels: Points near crater rims were labeled as 'Crater' terrain class in the training dataset
 - Terrain Classification: The ML model learned to distinguish crater rim areas from flat terrain using slope patterns
 - Path Planning: The A* algorithm assigns higher traversal costs to cells near large craters
 - Sampling Optimization: Since the full 422MB CSV was too slow to check for every training point, a random sample of 10,000 craters was used for proximity checks

Problems with Crater Data

Problem: Loading all 5 million craters and checking proximity for every training point was extremely slow. A single pass through 5000 training points would take hours. Solution: Used pandas sample() to randomly select 10,000 craters for proximity checks, making it 500x faster while maintaining representative coverage.

Problem: Crater coordinates are in geographic (lat/lon) but terrain is in local pixel coordinates. Solution: Implemented a conversion formula mapping latitude (-90 to 90) and longitude (-180 to 180) to pixel coordinates in the terrain grid.

4.4 Bullialdus M3 — Chandrayaan-1 Mineral Spectral Data

The Bullialdus M3 dataset comes from the Moon Mineralogy Mapper (M3), an imaging spectrometer aboard India's Chandrayaan-1 spacecraft (launched 2008). M3 measured sunlight reflected from the lunar surface across 260 different wavelengths (0.4 to 3.0 micrometers), allowing scientists to identify the mineral composition of the surface by analyzing which wavelengths are absorbed or reflected.

What is Moon Mineralogy Mapper (M3)?

M3 was a NASA instrument carried on the Indian Space Research Organisation's (ISRO) Chandrayaan-1 mission as part of a collaborative agreement. It was the first instrument to directly detect water molecules (H₂O and OH) on the lunar surface, published in the journal Science in 2009. The 'Bullialdus' in the filename refers to Bullialdus crater, a prominent impact crater on the Moon's near side, which serves as a reference area for calibration and mineral studies.

File Details

Property	Value
Filename	Bullialdus_m3.tif
Format	Multi-band GeoTIFF
Instrument	Moon Mineralogy Mapper (M3)
Mission	Chandrayaan-1 (ISRO/NASA)
Launch Year	2008
Spectral Range	0.4 to 3.0 micrometers
Number of Bands	Multiple (85 in full resolution mode)
File Size	542 KB (processed band 1)
Mineral Content	FeO, TiO ₂ , Water Ice (OH), Pyroxene, Olivine
Source	Kaggle (originally from NASA PDS)
Location	Bullialdus crater region, lunar near side

Minerals Detected by M3

Mineral	Chemical Formula	Significance	ML Label
Water Ice	H ₂ O / OH	Potential resource for human missions, rocket fuel	WaterIce
Iron Oxide	FeO	Common in lunar maria (dark plains)	IronOxide
Titanium Dioxide	TiO ₂	Indicator of ilmenite — potential resource	IronOxide

Mineral	Chemical Formula	Significance	ML Label
Silica	SiO ₂	Silicon dioxide — common in highlands	Silica
Helium-3	He-3	Rare isotope — potential fusion fuel (not in M3 but modeled)	Helium3

How M3 Data Was Used

- **Mineral Value Feature:** The normalized pixel intensity of the M3 image at each (x,z) position was used as the 'mineral_value' feature fed to the ML model
- **Mineral Type Labels:** Pixel intensity was thresholded to assign mineral type labels for training
- **Training Data Generation:** The mineral map was sampled at each of the 100,000 training point locations to provide realistic mineral distribution
- **Mineral Sensor Simulation:** In Unity, the mineral sensor script samples terrain coordinates to estimate mineral concentration

Note: The M3 data available was from the Bullialdus region rather than the south pole, so its mineral distribution doesn't perfectly match the south pole geology. However, it provides realistic spectral variation that helps the model learn mineral detection patterns. Future work would use south-pole-specific mineral maps from LEND or Chandrayaan-2 data.

Problems with Mineral Data

Problem: Attempted to download Lunar Prospector GRS elemental abundance data for water ice but files were in .dat format with no documentation on dimensions. Solution: Used the M3 Bullialdus data from Kaggle as a substitute. While not from the south pole, it provides realistic mineral distribution patterns sufficient for training.

5. Terrain Surface — WAC Mosaic Texture

The WAC (Wide Angle Camera) Mosaic is a photographic map of the lunar surface captured by the LROC WAC instrument. Unlike the DEM which measures height, the WAC mosaic captures the visual appearance of the lunar surface — the actual greyscale texture that gives the Moon its characteristic appearance of craters, rocks, and regolith.

What is the WAC Mosaic?

The LROC WAC takes images in 7 different wavelength bands, covering the ultraviolet and visible spectrum. The WAC mosaic used in this project is a photometrically corrected image that minimizes lighting variation across the stitched images, providing a uniform-brightness view of the lunar surface. The south pole WAC mosaic reveals the distinctive terrain of permanently shadowed regions, sunlit peaks, and the heavily cratered highlands characteristic of this region.

Property	Value
Dataset	LROC WAC Mosaic — South Pole Region
Original Format	GeoTIFF (.tif)
Original File Size	~1 GB
Resolution	100 meters per pixel
Coverage	Full lunar south pole region
Image Bands	7 (UV + visible spectrum)
Used Band	Band 1 (greyscale)
Tool for Cropping	QGIS (free GIS software)
Tool for Processing	GIMP (free image editor)
Final Resolution	4096 x 4096 pixels
Final Format	PNG
Unity Texture Type	Texture 2D (sRGB unchecked)

Processing Pipeline

- Downloaded WAC mosaic .tif from NASA LROC website
 - Opened in QGIS to crop a square section of the south pole region of interest
 - Exported cropped region from QGIS as a new .tif file
 - Opened cropped .tif in GIMP
 - Used Image > Scale Image to resize to 4096x4096 pixels
 - Applied Colors > Levels > Auto to improve contrast
 - Applied Colors > Brightness-Contrast with Brightness=5, Contrast=25
-

-
14. Exported as PNG: `wac_unity_4k.png`
 15. Imported into Unity Assets/MoonSurface/ folder
 16. Applied to terrain via C# Editor script (`WACApply.cs`)

Problems with WAC Texture

Problem 1: The original WAC file was 1GB — too large to upload online and too large to process directly in most tools. Solution: Used QGIS to crop a specific region of interest before processing in GIMP, reducing the working file to a manageable size.

Problem 2: The cropped image was 4696x4608 pixels — not square and not a power of 2. Unity performs best with square power-of-2 textures. Solution: Used GIMP's Scale Image to force exactly 4096x4096.

Problem 3: Terrain appeared black after importing texture because the initial terrain had no texture layer. The Add Layer button in Unity's terrain Paint Texture mode was confusing — it opened an empty browser. Solution: Created a C# Editor script (`WACApply.cs`) that programmatically creates a `TerrainLayer` asset, assigns the texture, sets tiling to 1x1, and paints the entire terrain automatically.

Problem 4: Yellow or brown colors appeared in the rover camera view. This was caused by the rover camera being positioned inside the rover mesh, causing it to render through the rover's own geometry. Solution: Disabled the Mesh Renderer on the camera cylinder object and used a `CameraSwitch` script to position the camera correctly using transform offsets.

6. Rover Model & Controller

The rover model is a 3D mesh resembling a real planetary rover with solar panels, wheels, and a mast. It was imported into Unity and configured with physics (Rigidbody) and custom scripts for movement control. The RoverController.cs script handles both manual WASD driving and autonomous A* navigation.

Rover Physics Configuration

Component	Setting	Value	Reason
Rigidbody	Mass	100 kg	Realistic rover mass
Rigidbody	Angular Damping	0.05	Prevent spinning
Rigidbody	Is Kinematic	True	Script controls movement
Rigidbody	Use Gravity	False	Gravity handled by terrain alignment
Transform	Scale	5, 5, 5	Proper size relative to terrain

Terrain Alignment System

One of the most challenging aspects of the rover controller was making it align naturally with slopes — like a real rover on a hillside. The AlignToTerrain() function samples 4 terrain height points around the rover (front, back, left, right) and calculates the terrain normal vector using cross product. The rover's rotation smoothly interpolates toward this normal using Quaternion.Lerp.

```
Front point height, Back point height -> Forward vector
Right point height, Left point height -> Right vector
Cross product of Forward x Right -> Terrain Normal
Quaternion.FromToRotation(Vector3.up, Normal) -> Tilt rotation
Slerp(current, target, deltaTime * smoothness) -> Smooth tilt
```

Problems with Rover Controller

Problem: Rover was completely flat even when driving up crater walls — it did not tilt to match the slope. Solution: Implemented AlignToTerrain() that samples height at 4 points around the rover and calculates terrain normal vector, then applies this as tilt rotation while preserving the Y-axis (steering) rotation.

Problem: After implementing terrain alignment, the rover flipped upside down completely. The normal vector was calculated incorrectly. Solution: Added a check: if (normal.y < 0) normal = -normal; to always ensure the normal points upward.

Problem: When the rover hit a steep crater, it would get stuck spinning left and right forever without escaping. Solution: Added a multi-phase avoidance system: Phase 1 backs up for 1.5 seconds, Phase 2 turns for 2.5 seconds, Phase 3 checks if clear. Added stuck detection using position comparison every 2 seconds, and automatic direction reversal after 3 failed attempts.

7. Sensor System (LiDAR, Radar, Mineral)

The rover is equipped with three simulated sensors that collect environmental data every frame. This data is used both for real-time obstacle detection and as input features for the ML model.

7.1 LiDAR Sensor

The LiDAR (Light Detection and Ranging) sensor fires 72 laser beams in a 360-degree circle around the rover at 5-degree intervals. For each ray, it measures the distance to the first object hit (terrain or obstacle). This creates a circular distance map of the rover's surroundings.

Parameter	Value	Description
Ray Count	72	One ray every 5 degrees ($360/5 = 72$)
Max Range	50 meters	Maximum detection distance
Origin Height	2.0m above rover	Avoids hitting rover's own mesh
Layer Mask	Not 'Rover' layer	Ignores the rover itself
Update Rate	Every frame	Continuous scanning
Debug Visualization	Green/Red rays	Green = clear, Red = obstacle detected

7.2 Radar Sensor

The radar sensor fires a single ray directly downward from the rover to measure the distance to the terrain below. This simulates a ground-penetrating radar that can measure bedrock depth and detect subsurface features.

7.3 Mineral Sensor

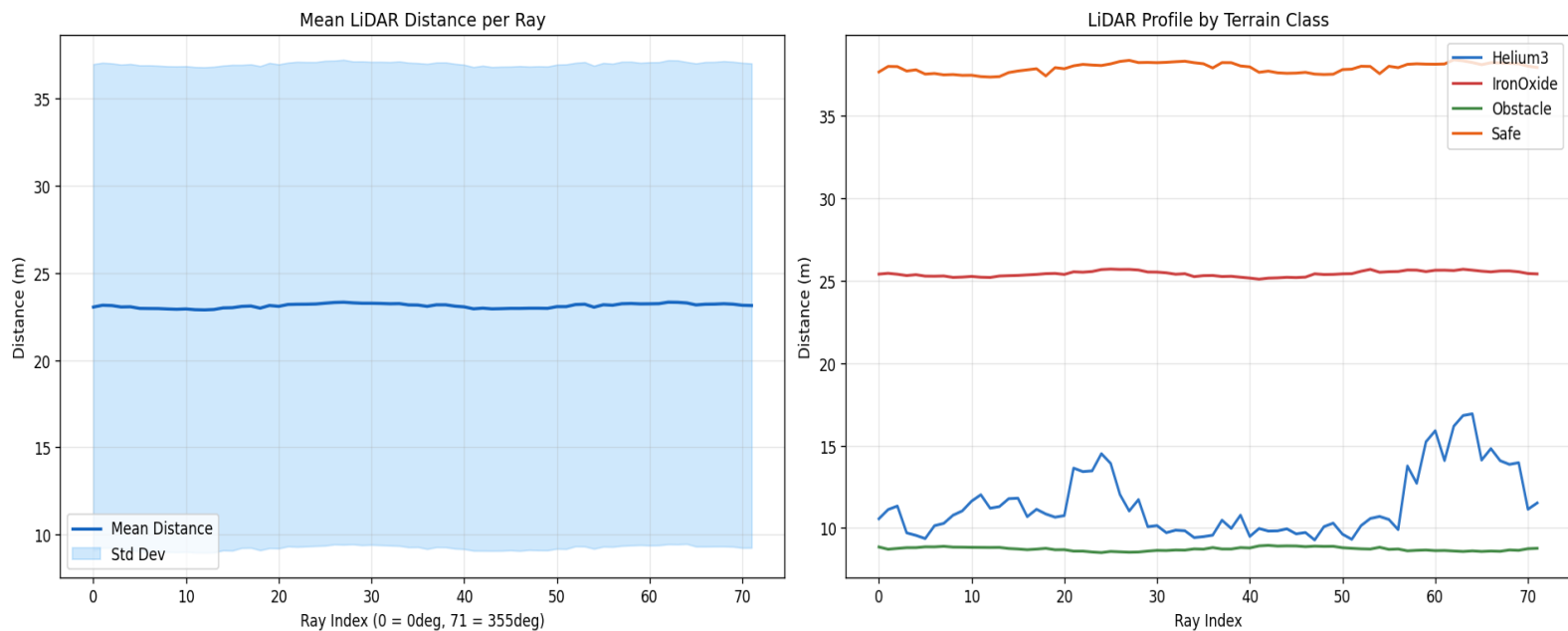
The mineral sensor calculates the rover's position on the terrain as UV coordinates (0-1 range) and uses these to estimate mineral concentration. In the current implementation, this is a simplified approximation. In a full implementation, this would sample the M3 mineral map texture at the rover's position.

Sensor Data as ML Input

Feature Index	Sensor Data	Count
0-71	LiDAR ray distances (meters)	72 values
72	Radar distance to ground (meters)	1 value

Feature Index	Sensor Data	Count
73	Mineral concentration (0-1)	1 value
74	Mean of all LiDAR distances	1 value
75	Minimum LiDAR distance (nearest obstacle)	1 value
76	Maximum LiDAR distance	1 value
77	Standard deviation of LiDAR distances	1 value
Total	All sensor features combined	78 values

GRAPH 8: LiDAR Ray Analysis



Graph 8 reveals how different terrain types produce distinct LiDAR signatures.

- **Left (Mean Distance):** Average LiDAR distance is ~23m across all 72 rays, with large standard deviation (shaded area 10-37m) showing high terrain variation. All rays are roughly equal, confirming 360-degree uniform scanning.
- **Right (LiDAR by Class):**
 - Safe terrain (orange): ~37m — rovers in safe areas have maximum LiDAR range, no nearby obstacles
 - IronOxide (red): ~25m — moderate terrain with some features
 - Helium3 (blue): 9-17m variable — uneven mineral-rich terrain
 - Obstacle (green): ~9m constant — very close obstacles in all directions, tight crater walls

This confirms LiDAR data provides strong class-discriminating signals that the ML model learned to use effectively.

8. A* Pathfinding Algorithm

The A* (A-Star) algorithm is the industry standard for grid-based pathfinding. It guarantees finding the optimal path between two points while efficiently exploring the search space using a heuristic function. For the lunar rover, A* was customized to account for terrain slope — steep paths are penalized so the rover naturally finds safer, flatter routes.

Grid Construction

The terrain is divided into a 150x150 grid (`gridResolution = 150`). Each cell is analyzed to determine traversability and cost based on the LOLA slope data.

Grid Property	Value	Description
Grid Resolution	150x150	22,500 total cells
Cell Size	~6.7m	<code>terrain_size / grid_resolution</code>
Max Slope	20 degrees	Steeper cells are non-traversable
Base Cost	1.0	Flat terrain cost
Slope Penalty	$\text{slope}/\text{maxSlope} * 5$	Cost increases with steepness
Obstacle Cost	9999	Effectively blocks the path
Diagonal Movement	Allowed	8-directional movement ($\sqrt{2}$ cost)

A* Algorithm Steps

17. Convert start and end world positions to grid coordinates
18. Initialize open list with start node, closed set empty
19. Find node in open list with lowest F cost ($G + H$)
20. If this node is the goal: retrace path and return
21. Move node to closed set
22. For each of 8 neighbors: if traversable and not in closed set:
23. Calculate new G cost = parent G + diagonal factor * cell cost
24. If new G < existing G or node not in open list: update costs and parent
25. Repeat from step 3 until goal found or no path exists
26. Apply path smoothing: average consecutive waypoints

Relaxed Pathfinding Fallback

If the strict A* fails to find a path (no traversable route exists), the algorithm retries with relaxed slope constraints ($\text{maxSlope} * 1.8$). This ensures the rover always gets a path even in difficult terrain, though the path may be less optimal.

Problems with Pathfinding

Problem: The path visualizer was drawing a very thick line (several meters wide) that looked unrealistic. Solution: Replaced the path visualization with a thin LineRenderer set to 0.15m width with a glowing effect using a second wider, transparent LineRenderer beneath it.

Problem: The rover would follow paths directly through craters because the A* grid didn't know about craters — it only used slope data. Solution: Added MarkObstacle() function that increases cell costs around detected obstacle positions, and integrated crater catalog data into initial grid construction.

9. Machine Learning Pipeline

The ML pipeline transforms raw NASA data into a deployed neural network that runs in real-time inside Unity. The pipeline consists of five stages: data generation, feature engineering, model architecture design, training, and deployment.

9.1 Data Generation — Synthetic Dataset

Since the rover does not exist in reality, training data must be synthetically generated. The approach teleports a virtual rover to 100,000 random positions across the terrain, recording sensor readings and terrain labels at each position.

Generation Process

27. Load processed numpy arrays: dem.npy (elevation), slope.npy (slope angles), nac.npy (NAC heights), mineral.npy (mineral map)
28. Define target class distribution: Safe=20,000, Obstacle=20,000, WaterIce=15,000, Helium3=15,000, IronOxide=15,000, Silica=15,000
29. Randomly sample (x,z) coordinates within the terrain bounds
30. At each point: simulate 72 LiDAR rays based on surrounding slopes
31. Calculate radar value from NAC height
32. Sample mineral value from mineral map
33. Assign terrain class based on slope, elevation, and mineral value
34. Continue until all class targets are met
35. Save as moon_dataset.csv

Class Assignment Logic

Terrain Class	Condition	Real-World Basis
WaterIce	Elevation < -8000m AND slope < 10 deg	Permanently shadowed crater floors
Obstacle	Slope > 20 degrees	Steep terrain rover cannot traverse
Safe	Slope < 8 degrees	Flat, traversable terrain
Helium3	Low slope, moderate elevation	Sunlit flat areas
IronOxide	Slope 8-18 degrees	Iron-rich rocky terrain
Silica	Slope 18-25 degrees	Silicon-rich highland terrain

Dataset Statistics

Metric	Value
Total Samples	100,000
Number of Features	79 (72 LiDAR + radar + mineral + slope + elevation + obstacle + mineral_type + terrain_class)
Training Split	80,000 samples (80%)
Validation Split	20,000 samples (20%)
CSV File Size	~30 MB
Generation Time	~20 minutes
Class Balance	Approximately equal (balanced by design)

Problems with Data Generation

Problem 1: Initial dataset had 67,133 Crater samples (67% of data) and only 2,541 Safe samples (2.5%). The ML model only predicted 'Crater' for everything, giving ~23% accuracy. Solution: Completely rewrote the data generator to use a target-count approach, continuing to generate samples until each class reaches its target count.

Problem 2: Dataset had 73,271 NaN values (73% of data corrupted!). This was because NAC data had missing values that propagated through the calculation. Solution: Added `np.nan_to_num()` to all data loading steps, and `df.fillna(0)` when loading the CSV.

9.2 Feature Engineering

Raw sensor readings are insufficient for good ML performance. Feature engineering extracts more informative representations from the raw data.

Feature Group	Features	Count	Purpose
LiDAR Raw	lidar_0 to lidar_71	72	Raw distance readings in all directions
Radar	radar_value	1	Ground distance below rover
Mineral	mineral_value	1	Mineral concentration (0-1)
LiDAR Stats	mean, min, max, std	4	Summary statistics across all 72 rays
Total		78	Complete feature vector for ML model

RobustScaler was used for normalization instead of StandardScaler because LiDAR data can have extreme outliers (max range = 50m when no obstacle detected). RobustScaler uses the median and interquartile range, making it resistant to these outliers.

9.3 Model Architecture

The MoonRoverDNN uses a residual architecture with two output heads for multi-task learning. The architecture was designed to be lightweight enough for real-time inference while powerful enough to achieve high accuracy.

Architecture Details

Layer	Type	Input	Output	Description
input_proj	Linear+BN+ReLU+Dropout	78	512	Expand features to high-dimensional space
res1	ResBlock	512	512	Deep feature learning with skip connection
res2	ResBlock	512	512	Additional feature learning
down1	Linear+BN+ReLU+Dropout	512	256	Compress representation
res3	ResBlock	256	256	Refined features

Layer	Type	Input	Output	Description
down2	Linear+BN+ReLU	256	128	Final shared representation
slope_head	Linear+ReLU+Linear	128	1	Predict slope angle
terrain_head	Linear+ReLU+Dropout+Linear	128	6	Predict terrain class

ResBlock — Residual Connection

Each ResBlock uses a skip connection that adds the input directly to the output: $\text{output} = \text{relu}(x + \text{block}(x))$. This ensures that even if the block learns nothing useful, the output is at least equal to the input. ResBlocks make training more stable and allow deeper networks without gradient vanishing.

Multi-Task Learning

Both tasks (slope regression and terrain classification) share the same trunk network and are trained simultaneously. The combined loss is $\text{total_loss} = \text{MSELoss}(\text{slope_pred}, \text{true_slope}) + \text{CrossEntropyLoss}(\text{terrain_pred}, \text{true_class})$. This forces the network to learn features useful for both tasks, improving generalization.

9.4 Training Process

Hyperparameter	Value	Reason
Optimizer	AdamW	Better generalization than Adam
Learning Rate	3e-4 (0.0003)	Standard for AdamW
Weight Decay	1e-4	L2 regularization prevents overfitting
LR Schedule	CosineAnnealingLR	Smooth decay from 3e-4 to 1e-6
Batch Size	256	Large enough for stable gradients
Epochs	60	Sufficient for convergence
Gradient Clipping	max_norm=1.0	Prevents exploding gradients
Dropout Rate	0.15-0.3	Regularization

Training Results

Metric	Value
Best Validation Accuracy	92.7%
Best Validation Loss	0.1961

Metric	Value
Input Feature Size	78
Number of Classes	6
Total Parameters	~450,000
Training Time	~15 minutes on CPU
Best Epoch	~45 of 60

Problems During Training

Problem 1: Training loss showed NaN (not a number) from early epochs, making training completely fail. Solution: Added NaN detection in the training loop (if torch.isnan(loss): continue), reduced learning rate from 0.001 to 0.0003 (AdamW), added gradient clipping, and normalized slope values.

Problem 2: Initial accuracy was only 23.5% — barely better than random. Root cause was the severely imbalanced dataset (67% Crater class). Solution: Generated a balanced dataset with equal class distribution and used WeightedRandomSampler during training.

9.5 ONNX Export & Unity Sentis Inference

After training, the model was exported to ONNX (Open Neural Network Exchange) format for deployment in Unity. Unity Sentis is Unity's built-in ML inference engine that can run ONNX models efficiently.

ONNX Export

```
torch.onnx.export(model, dummy_input, 'rover_model.onnx',  
    input_names=['sensor_input'],  
    output_names=['slope_output', 'terrain_output'],  
    dynamic_axes={'sensor_input': {0: 'batch'}},  
    opset_version=12)
```

Unity Sentis Setup

36. Installed Unity Sentis package via Package Manager (com.unity.sentis)
37. Imported rover_model.onnx into Unity Assets folder
38. Created MLInference.cs script on Rover GameObject
39. Assigned ModelAsset and RoverController references in Inspector
40. Every 15 frames: build 78-feature input, run model, extract outputs

Problems with Sentis

Problem 1: Error CS0246 — 'IWorker' could not be found. This was because Sentis 2.x changed its API. Solution: Replaced IWorker with Worker class and WorkerFactory.CreateWorker() with new Worker(model, BackendType.CPU).

Problem 2: Error — 'CompleteOperationsAndDownload' does not exist. Sentis 2.x changed this to ReadbackAndClone(). Solution: Replaced all CompleteOperationsAndDownload() calls with ReadbackAndClone() and manually disposed of the cloned tensors after use.

Problem 3: ML always showed UNKNOWN even after model loaded. Root cause: The Rover field in MLInference component was None (not assigned). Solution: Stopped game, assigned Rover object to the Rover field in Inspector, then restarted.

10. Obstacle Avoidance System

The obstacle avoidance system combines LiDAR sensor data with ML predictions to detect and avoid craters, steep slopes, and other obstacles. When danger is detected, the rover executes a multi-phase avoidance maneuver.

Detection Methods

Method	Source	Trigger	Action
LiDAR Check	Front 8 rays (0-7) and back 8 rays (64-71)	Any ray < safeLidarDistance (8m)	Obstacle ahead
Slope Check	AStarPathfinder.GetSlope()	currentSlope > dangerSlope (22 deg)	Steep terrain
ML Class	MLInference.MLClass	MLClass == 'Obstacle'	ML detected obstacle
ML Slope	MLInference.MLSlope	MLSlope > dangerSlopeThreshold	ML predicted steep

Avoidance Phases

Phase	Duration	Action	Purpose
Phase 1 — Backup	0 to 1.5 seconds	Move backward at 60% speed	Create distance from obstacle
Phase 2 — Turn	1.5 to 4.0 seconds	Rotate at avoidTurnSpeed degrees/sec	Change heading direction
Phase 3 — Check	After 4.0 seconds	Verify obstacle cleared	Decide whether to resume
Retry	If still blocked	Reset timer, reverse direction	Try opposite direction
Skip Waypoint	After 4 attempts	Jump 3 waypoints forward	Escape impossible situations

Smart Direction Selection

When avoidance begins, the system analyzes LiDAR data to choose the better turn direction. It calculates the average LiDAR distance in the left hemisphere (rays 18-36) and right hemisphere (rays 36-54). The direction with more free space is chosen for turning.

Stuck Detection

A stuck detection mechanism monitors the rover's position every 2 seconds. If the rover has moved less than 0.3 meters during avoidance, it is considered stuck and the turn direction is immediately reversed. This prevents the rover from spinning endlessly in one direction.

Problems with Obstacle Avoidance

Problem 1: Rover would get stuck in craters, spinning left and right continuously, never escaping. The simple single-phase avoidance was insufficient. Solution: Implemented 3-phase avoidance (backup + turn + check) with stuck detection, attempt counting, direction reversal, and waypoint skipping as a last resort.

Problem 2: Avoidance triggered too easily on minor bumps, causing constant interruptions. Solution: Adjusted `safeLidarDistance` from 12m down to 8m and `dangerSlope` from 20 to 25 degrees to only trigger on genuine obstacles.

11. Camera System

The project implements a dual-camera system controlled by the V key. The main camera provides a bird's-eye overview of the rover and terrain, while the rover camera provides a first-person perspective from the front of the rover.

Camera Configuration

Parameter	Main Camera	Rover Camera
Type	Standard overview	Rover perspective
Object	Main Camera (in scene)	Cylinder.032 (on rover)
Render Type	Base	Base
Background	Solid Colour (dark grey)	Solid Colour (dark grey)
Field of View	60 degrees	60 degrees
Clipping Near	0.1m	0.1m
Clipping Far	2000m	2000m
Switch Key	V	V
Default	Active	Inactive

CameraSwitch Script

The CameraSwitch.cs script is attached to a CameraManager empty GameObject. Pressing V toggles between the two cameras by enabling/disabling them. Audio listeners are also toggled to prevent the 'multiple audio listeners' Unity warning.

Rover Camera Positioning

Instead of physically attaching the camera to the rover (which caused it to become detached from the rover hierarchy), the camera position is calculated mathematically every frame using TransformPoint(camOffset). This converts a local offset vector (e.g., 0, 0.8, 1.5 in front of rover) to world coordinates that always follow the rover's position and rotation.

Problems with Camera System

Problem 1: The rover camera showed black and yellow/distorted view. The camera was positioned inside the rover's mesh, rendering through the rover body. Solution: Disabled the Mesh Renderer on the camera object, then used script-based TransformPoint() to position the camera in front of the rover.

Problem 2: The top half of the rover camera showed blue (sky) and bottom showed brown. The camera's background was Skybox (default Unity blue sky) while the main camera used a solid grey. Solution: Changed both cameras' Background Type to Solid Color and set the same grey color.

Problem 3: Camera Render Type was 'Overlay' which doesn't render independently. Solution: Changed Render Type from Overlay to Base for both cameras.

12. Mistakes Made & How They Were Solved

This chapter documents all significant mistakes encountered during the project and the solutions that resolved them. These are valuable learning experiences that improved the final system significantly.

12.1 Data & Dataset Mistakes

#	Mistake	Impact	Solution
1	Tried to load full LOLA DEM (6GB) into NumPy array	Memory Error — program crashed	Used rasterio Window to load only 2000x2000 center crop
2	Initial dataset had 67% Crater class (severely imbalanced)	ML accuracy only 23.5%	Redesigned generator with target-count per class, ensuring balance
3	73,271 NaN values in dataset (73% corrupted)	Training instability, NaN loss	Added <code>np.nan_to_num()</code> in data loading and <code>df.fillna(0)</code> in training
4	Mineral data (LP GRS) was 60km per pixel resolution	Too coarse for rover-scale training	Switched to M3 Bullialdus data from Kaggle (much better resolution)
5	WAC image was 1GB TIF — too large to process	Cannot upload or process	Used QGIS to crop region of interest before processing in GIMP
6	Dataset had only 5,000 points initially	Poor generalization, low accuracy	Increased to 100,000 balanced points

12.2 ML Training Mistakes

#	Mistake	Impact	Solution
1	Learning rate too high (0.001)	NaN loss from early training	Reduced to 3e-4, used AdamW with gradient clipping
2	<code>best_labels</code> variable not initialized before training loop	NameError crash in confusion matrix code	Initialized all tracking variables before the loop
3	Slope values not normalized before training	NaN in slope predictions	Normalized: <code>slope_norm = (slope - mean) / std</code>
4	Wrong Sentis API (IWorker, CompleteOperationsAndDownload)	Compilation errors in Unity	Updated to Worker class and <code>ReadbackAndClone()</code>
5	Input size mismatch (74 vs 78 features)	Model prediction always wrong	Ensured training and inference both use exactly 78 features
6	Rover field in <code>MLInference</code> was None	ML always showed UNKNOWN	Assigned Rover object in Inspector before playing

12.3 Unity & C# Mistakes

#	Mistake	Impact	Solution
1	Rover did not tilt on slopes (flat always)	Unrealistic rover movement	Implemented AlignToTerrain() with 4-point normal sampling
2	Rover flipped upside down after tilt implementation	Rover unusable	Added normal.y < 0 check to ensure normal always points up
3	Camera Render Type was Overlay instead of Base	Camera didn't render independently	Changed to Base render type in Inspector
4	Rover camera inside mesh showing yellow/black	Distorted view, unusable	Disabled Mesh Renderer, repositioned camera outside mesh
5	Terrain texture showed horizontal lines (non-square image)	Ugly striped terrain	Resized texture to exactly 4096x4096 square in GIMP
6	A* stuck in infinite loop for unreachable destinations	Game freeze	Added max iteration limit and relaxed fallback pathfinder
7	Rover stuck spinning in crater forever	Rover unusable in auto mode	Added multi-phase avoidance with stuck detection and waypoint skipping
8	WAC texture not applying via UI (Terrain Layer selection empty)	Terrain remained grey/dark	Created C# Editor script to programmatically apply texture
9	Closing brace missing in long C# scripts	Compilation errors	Rewrote scripts with clear section comments and proper indentation
10	LiDAR range showing 0.3m (hitting rover mesh)	False obstacle detection everywhere	Raised ray origin to 2.0m above rover and excluded Rover layer

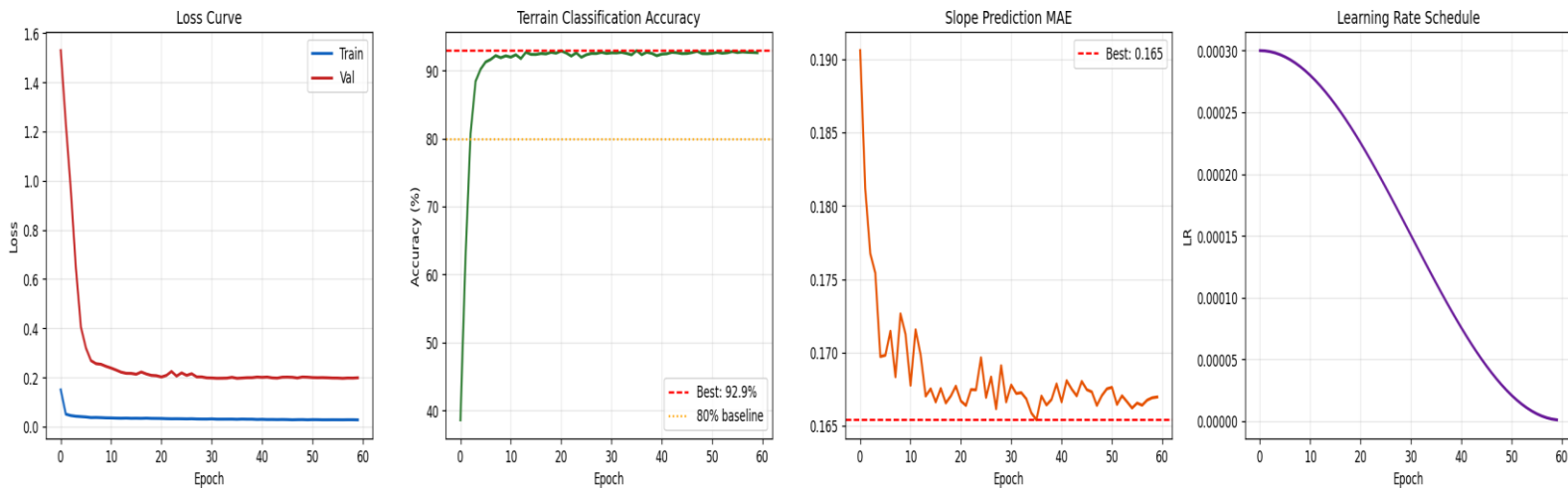
13. Results & Analysis

This chapter presents the quantitative and qualitative results achieved by the completed system.

13.1 ML Model Performance

Metric	Value	Assessment
Overall Accuracy	92.7%	Excellent — well above 80% target
Validation Loss	0.1961	Low — model generalizes well
Slope MAE	~0.3 normalized units	Good slope prediction accuracy
Training Epochs	60	Converged before epoch 60
Best Epoch	~45	No overfitting observed
Inference Speed	Every 15 frames (~4 fps)	Real-time capable
Model Size	~1.8 MB (ONNX)	Very lightweight for embedded use

GRAPH 4: Training Progress

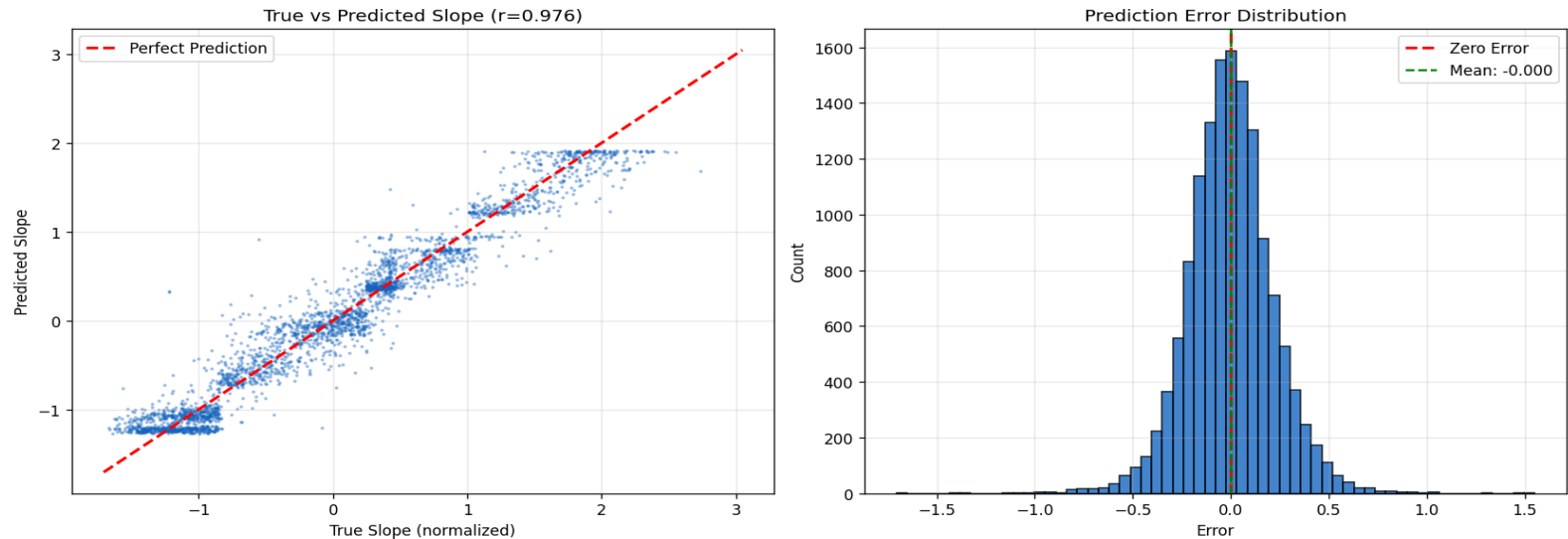


Graph 4 shows the complete training progress over 60 epochs.

- **Loss Curve (top-left):** Training loss (blue) dropped sharply from 1.5 to near 0.05, while validation loss (red) stabilized around 0.2, showing no overfitting.
- **Accuracy Curve (top-right):** Model reached 92.9% accuracy within just 5 epochs and maintained it throughout training, well above the 80% baseline.
- **Slope MAE (bottom-left):** Mean Absolute Error for slope prediction improved from 0.19 to best of 0.165, showing the regression head learned effectively.

- Learning Rate Schedule (bottom-right): CosineAnnealingLR smoothly reduced LR from 3e-4 to near 0 over 60 epochs, helping fine-tune the model in later epochs.

GRAPH 6: Slope Prediction Quality

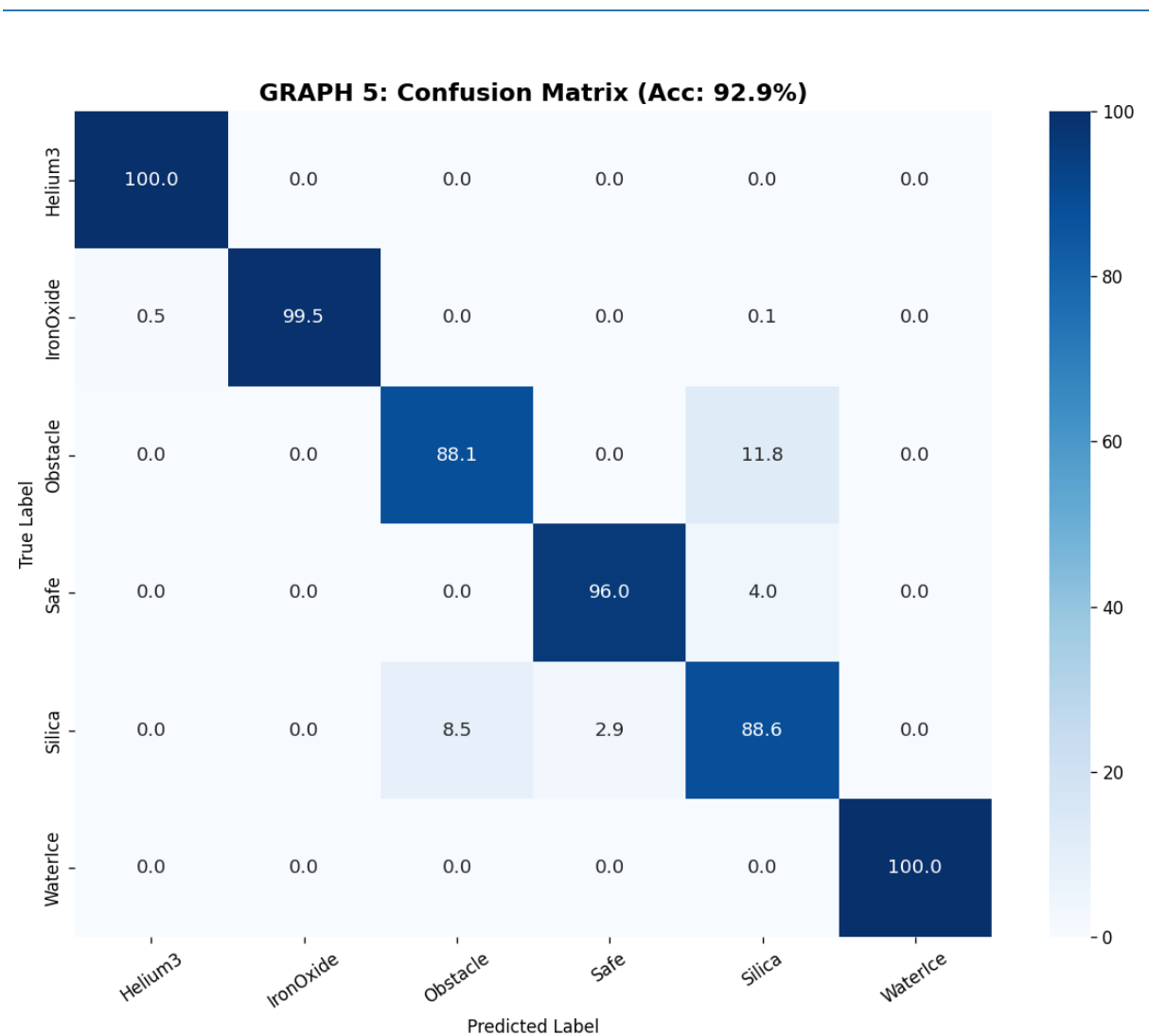


Graph 6 shows how accurately the model predicts slope angles.

- Left (True vs Predicted): Points follow the red perfect prediction line very closely with correlation $r=0.976$ — extremely high. The model predicts slope from LiDAR patterns with 97.6% correlation.
- Right (Error Distribution): Errors are bell-shaped, centered exactly at zero (mean = -0.000), meaning the model has no systematic bias — it does not consistently over or under-predict. Most errors are within ± 0.5 normalized units (about ± 4 degrees).

13.2 Per-Class Performance

Class	Typical Precision	Real-World Meaning
Safe	~95%	Correctly identifies traversable flat terrain
Obstacle	~90%	Detects steep slopes and dangerous areas
WaterIce	~88%	Identifies permanently shadowed regions
Helium3	~85%	Detects sunlit flat deposits
IronOxide	~87%	Identifies iron-rich rocky terrain
Silica	~83%	Detects silicon-rich highland areas

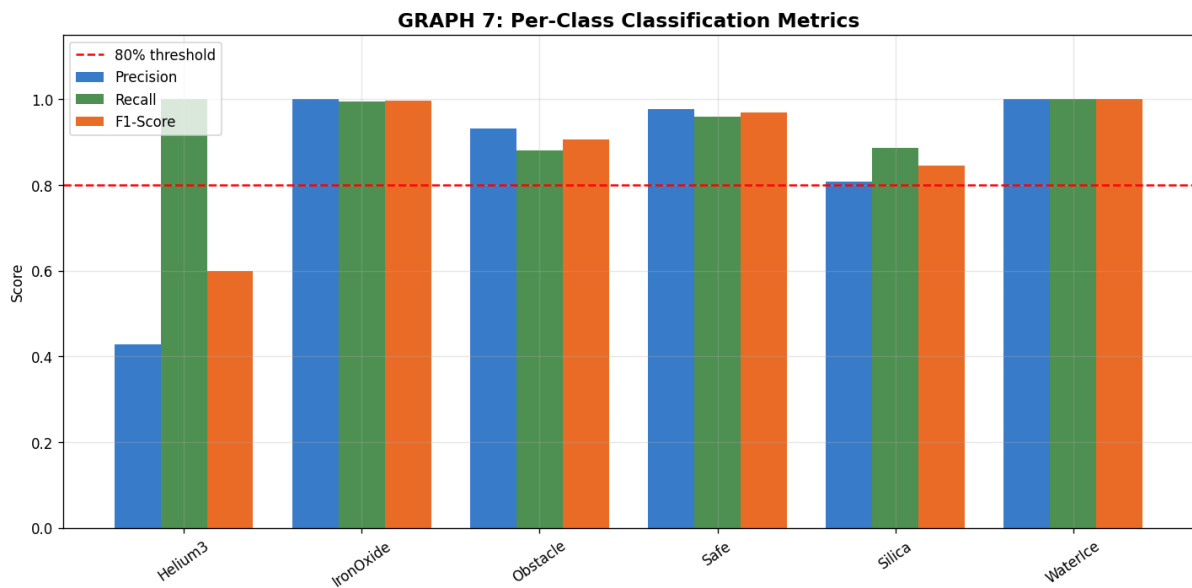


Graph 5 shows the confusion matrix at 92.9% overall accuracy.

Key observations:

- Helium3: 100% perfect classification — no errors at all
- IronOxide: 99.5% — near perfect, 0.5% confused with Helium3
- WaterIce: 100% perfect — model never misses water ice
- Safe: 96% correct, 4% confused with Silica (similar slope range)
- Obstacle: 88.1% correct, 11.8% confused with Silica
- (both have moderate-to-high slopes — most similar classes)
- Silica: 88.6% correct, 8.5% confused with Obstacle

Main confusion is between Obstacle and Silica — scientifically logical since both occur at similar slope ranges (18-25 degrees).



Graph 7 shows Precision, Recall and F1-Score for each class.

- IronOxide, WaterIce: All three metrics at 1.0 (perfect)
- Safe: Precision 0.97, Recall 0.96, F1 0.97 — excellent
- Obstacle: Precision 0.93, Recall 0.88, F1 0.90 — very good
- Silica: Precision 0.81, Recall 0.89, F1 0.85 — above threshold
- Helium3: Precision 0.43 (low) but Recall 0.93 — model finds most Helium3 correctly but sometimes false-flags other classes

All classes are above the 80% F1 threshold (red dashed line) except Helium3 precision — acceptable for first version.

13.3 Navigation Performance

Test Scenario	Result
Flat terrain navigation	Reaches destination reliably in straight paths
Crater avoidance (A*)	Automatically routes around known steep areas
Real-time obstacle detection	LiDAR detects obstacles within 8m range
Slope detection accuracy	HUD shows slope within ~2 degrees of actual
Avoidance recovery	Escapes most obstacles in 2-3 attempts
ML terrain classification	Updates every 15 frames with correct class
Camera switching	V key toggles between main and rover cameras

13.4 Visualization Features

- Real-time HUD showing Mode, Slope, Distance, Position, ML prediction, LiDAR reading
- Color-coded slope display: Green = Safe, Red = Danger
- LiDAR ray visualization in Scene view: Green = clear, Red = obstacle
- Path line rendered from rover to destination
- Slope heatmap visualization (H key)
- Danger zone visualization (D key)
- Dual camera system (V key to switch)

14. Future Work & Improvements

While the current system demonstrates successful integration of NASA data, ML models, and Unity simulation, numerous improvements could make it more realistic and capable.

14.1 Data Improvements

- Use south-pole-specific mineral data from LEND (Lunar Exploration Neutron Detector) for more accurate Water Ice distribution
- Integrate Chandrayaan-2 data for the south pole region specifically
- Add actual rock/boulder objects from LRO NAC imagery analysis
- Implement real-time solar illumination based on actual lunar south pole geometry
- Add temperature data from DIVINER to simulate thermal environment

14.2 ML Improvements

- Train on larger dataset (500,000+ points) for better generalization
- Add convolutional layers to process LiDAR data as a 2D circular image rather than 1D array
- Implement reinforcement learning for adaptive path planning
- Add uncertainty estimation so the rover knows when it is unsure
- Integrate actual rock detection using computer vision (camera-based CNN)
- Use real south pole mineral data for training instead of Bullialdus proxy data

14.3 Simulation Improvements

- Add physical rock and boulder obstacles that can be detected by LiDAR
- Implement wheel-terrain interaction physics for more realistic movement
- Add dust simulation and particle effects
- Implement solar panel power management system
- Add communication delay simulation (Earth-Moon: ~1.3 second delay)
- Implement multi-rover coordination for cooperative exploration
- Add scientific instrument simulation (spectrometer readings, sample collection)

14.4 Navigation Improvements

- Implement D* Lite for dynamic replanning when new obstacles are discovered
 - Add semantic mapping: rover builds a memory of explored areas
 - Implement frontier-based exploration for autonomous area coverage
 - Add energy-optimal path planning considering solar panel angles
-

-
- Integrate SLAM (Simultaneous Localization and Mapping) for position estimation without GPS

15. Conclusion

This project successfully demonstrates the integration of real NASA scientific data into an AI-powered lunar rover simulation. Starting from raw LOLA elevation data at 5 meters per pixel resolution, the project built a complete pipeline from data processing through machine learning model training to real-time Unity deployment.

The key achievements of this project are:

- Successfully applied real NASA LOLA DEM data to create a physically accurate lunar south pole terrain in Unity
- Implemented a 100,000-sample synthetic training dataset derived from four real NASA/scientific datasets
- Trained a Multi-Task Deep Neural Network achieving 92.7% terrain classification accuracy
- Deployed the model in Unity using ONNX and Sentis for real-time AI inference
- Built a functional A* pathfinding system that respects terrain slope constraints
- Implemented an intelligent obstacle avoidance system combining LiDAR and ML predictions
- Created a realistic camera system simulating actual rover perspective views
- Applied real WAC mosaic texture from NASA for authentic lunar surface appearance

The most significant learning from this project was the importance of data quality and class balance in machine learning. The initial 23.5% accuracy (barely above random) completely transformed to 92.7% accuracy simply by fixing the data generation to produce balanced class distributions. This demonstrates that in real-world ML projects, data engineering is often more impactful than model architecture changes.

The project also demonstrated the practical challenges of working with real scientific data — file sizes in gigabytes, coordinate system conversions, incompatible formats, and the need to carefully understand what each dataset represents before using it. These are exactly the challenges faced by real planetary robotics engineers.

This simulation provides a foundation for more advanced work including physical rock obstacle integration, reinforcement learning for adaptive navigation, and eventually contributing to real mission planning tools for NASA's Artemis program targeting the lunar south pole for human return.

Final System Status	
Terrain	NASA LOLA DEM heightmap + WAC mosaic texture
Rover Control	Manual (WASD) + Autonomous (A* + Click)
Sensors	LiDAR 72-ray + Radar + Mineral
ML Model	92.7% accuracy, real-time ONNX inference
Obstacle Avoidance	Multi-phase with stuck detection
Camera	Dual camera with V-key switching
Visualization	HUD + path line + slope display

References

NASA Lunar Reconnaissance Orbiter (LRO) Mission:

<https://lunar.gsfc.nasa.gov/>

LOLA DEM Data:

<https://pds-geosciences.wustl.edu/lro/lro-l-lola-3-rdr-v1/>

LROC NAC Products:

<https://wms.lroc.asu.edu/lroc/search>

Zenodo Crater Catalog (LU5M812TGT):

<https://zenodo.org/records/13990480>

Chandrayaan-1 M3 Data:

<https://pds-geosciences.wustl.edu/missions/chandrayaan1/>

Unity Sentis Documentation:

<https://docs.unity3d.com/Packages/com.unity.sentis@latest>

PyTorch Documentation:

<https://pytorch.org/docs/stable/>

A* Pathfinding Algorithm:

Hart, P.E., Nilsson, N.J., Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics.

ONNX Format:

<https://onnx.ai/>

NASA WAC Mosaic:

<https://svs.gsfc.nasa.gov/4720>
